DinkData rebuild plan

Oct 9, 2026 · @Marti

Don't build the new app on top of DinkData. Keep its trained pickleball ball tracker and its labelled clips, then build a new pipeline that turns each video into a timeline of events (serves, hits, bounces, rally ends) tied to players. Every stat is computed from that timeline, and a quick human review step keeps the numbers trustworthy while the models improve.

What DinkData is today

DinkData is a fork of TrackNetV3, a research model for tracking badminton shuttlecocks, retrained to find the pickleball. About 1,400 of its ~5,000 lines are new pickleball code (5 commits, Apr–Jul 2025). It outputs one thing: the ball's pixel position per frame. It has no player detection, no hit or bounce detection, no scoring and no web app.

Part	What it does	Verdict
`model.py`, `train.py`, `test.py`, `predict.py`, `dataset.py`	TrackNet heatmap model finds the ball in 8-frame windows. InpaintNet fills short gaps where the ball is hidden.	Keep. This is the most valuable piece. Repackage it as one inference module.
`create_gt_csv.py`	Converts CVAT ball annotations (1280×720 rally clips) into per-frame `Frame, Visibility, X, Y` CSVs.	Keep the labels. Re-use the script for new labelling.
`rally_segment_features.py` + `rally_segmentation.ipynb`	Finds rallies from ball speed (>5 px/frame, ≥3 s long), counts net crossings with a fixed pixel band, then cuts a highlight reel with ffmpeg.	Rework. Good first heuristic, but it uses pixel units and hand-tuned per-video values.

Part	What it does	Verdict
`keypoint_test.py`	Click 12 points on one frame (4 corners, 4 kitchen-line ends, 4 centre-line points) and compute a homography to a 20×44 ft top-down court. The kitchen lines are in the template, but nothing else in the repo uses them.	Keep the court template. Replace the clicks with auto-detection plus a 4-click fallback.
`error_analysis.py`, `correct_label.py`	Dash tools from upstream to inspect tracking errors and correct labels.	Use for model debugging only. Not product code.
`preprocess.py`, `generate_mask_data.py`	Extract frames to PNG, build median backgrounds and splits.	Rework. Splits leak data (see below).

What is not in the repo: the trained checkpoints (`ckpts/`), the labelled videos (`data/`) and the homography files (`keypoints/`). All three are git-ignored, so confirm someone still has them.

Problems that block reuse

The repo does not run on a fresh clone, and two of its measurements are wrong by design. Fix these before trusting any number from it.

1. **Crashes on import.** `utils/general.py` loads `keypoints\court_base.png` and `homography.npy` when imported. The files are git-ignored, so `cv2.imread` returns `None` and every script fails.
2. **Windows-only paths.** Backslash paths are hardcoded (`predict.py` splits on `\\` , the notebook paths too). It breaks on Mac, Linux and cloud GPUs.
3. **Stale dependencies.** `requirements.txt` pins torch 1.10, but `train.py` calls `torch.load(..., weights_only=False)` , which that version does not accept.

4. **Optimistic accuracy.** Validation and test rallies come from the same matches as training (the last rallies of each match). Scores will drop on new courts, lighting and players. Hold out whole matches or venues instead.
5. **Airborne ball projected onto the court.** `make_inset` maps every ball position through the floor homography. That is only correct at the bounce. In flight, the ball lands on the minimap metres away from its real spot. (The minimap call is currently commented out.)
6. **Speeds in pixels.** Rally speed and the "active" threshold use px/frame. A far-court ball covers fewer pixels than a near-court one at the same speed, so the far team always looks slower.
7. **Per-video magic numbers.** Net band (`NET_Y_UPPER=145` , `NET_Y_LOWER=300`), speed threshold and gap sizes are tuned by hand for one video. They should come from the court calibration.
8. **No tests, no package, no CI.** Logic lives partly in notebooks with commented-out blocks.
9. Mislabelled calibration clicks. From point 5 on, the click order in the keypoint_test.py docstring does not match the template the code fits to (it lists "far-right kitchen" where the code expects far-left, and skips one point). The window only says "Click point N/12", so following the docstring gives a wrong homography.

Stats you can compute accurately

A single fixed camera gives accurate positions only for things touching the floor: player feet and ball bounces. Build stats on those first. Anything that needs the ball's height or true speed in the air is an estimate, so label it that way.

Every stat below comes from four detected signals: **rally start and end**, **hits** (who hit, when, volley or after a bounce), **bounces** (where on the court) and **player positions**. Confidence assumes a 1080p, 30 fps fixed camera with all four corners in view.

Stat (per player)	How it is computed	Confidence	Phase
Rallies played, rally length (shots, seconds)	Rally segmenter on ball track + hit count	High	1

Stat (per player)	How it is computed	Confidence	Phase
Points won and lost, side-outs	Rally ending (net, out, double bounce, unreturned) + side-out scoring state machine checks the result	High with review, medium fully automatic	1
Serves in %, faults, serve depth	First bounce after the serve, mapped to the court	High	1
Return in %, return depth	Second hit of the rally + its bounce	High	1
Court coverage heatmap, distance moved	Player foot points through the homography	High	1
Shots hit, share of team shots	Hit events attributed to the nearest player's wrist or paddle	Medium to high	2
Errors: into net, out	Last hit of a lost rally; ball fails to cross the net, or first bounce lands outside the lines	High for net, medium for out	2
Third-shot drop %, drive % and lob %	Serving team's 3rd shot classed by speed, arc and where it bounces. A drop succeeds if it lands in the kitchen (or is taken below net height) and is not attacked next shot	Medium	2
Kitchen arrival %	Feet within ~2 ft of the kitchen line by the opponent's next hit after your serve, return or drop	High	2
Dinks, dink errors, dink rally length	Soft shots from the kitchen line that bounce in the opponent's kitchen	Medium	2
Volleys vs groundstrokes	Hit with or without a bounce since the last hit	Medium to high	2

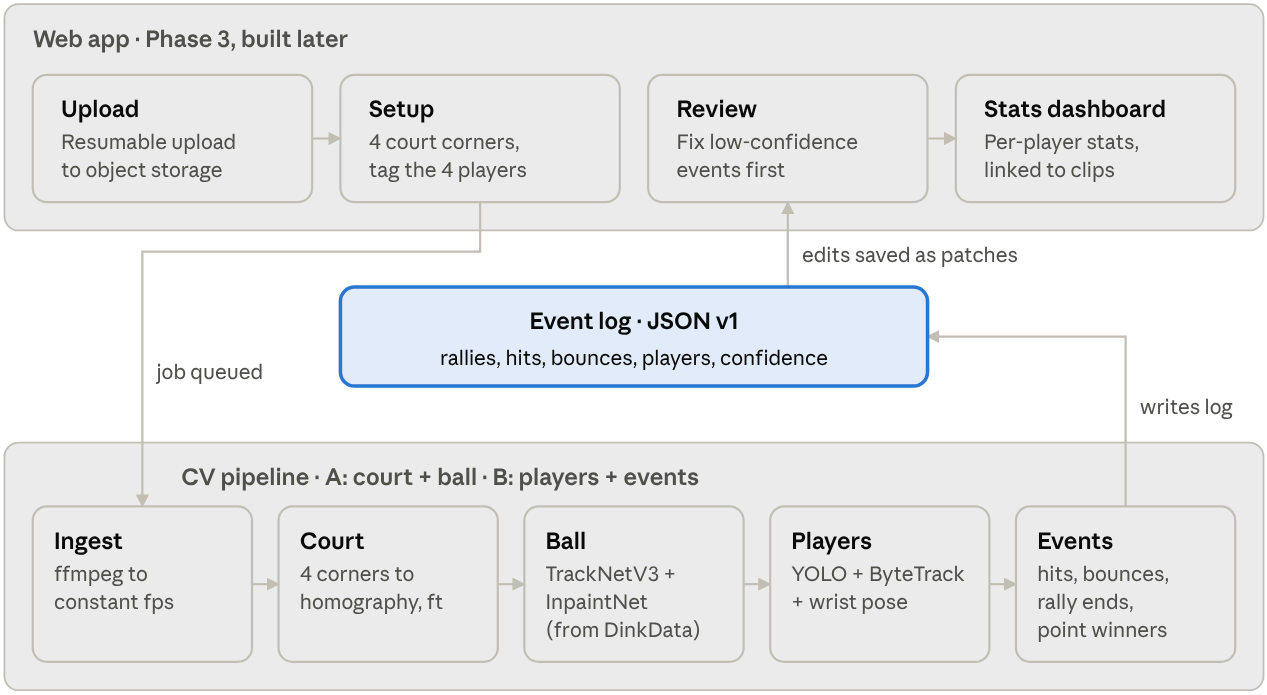
Stat (per player)	How it is computed	Confidence	Phase
Winners	Last hit of a won rally that the opponents never touched	Medium	2
Speed-ups attempted and won	Sudden speed jump inside a dink exchange	Medium	3
Forehand vs backhand	Pose: which side of the body the paddle is on at contact	Medium	3
Ball speed (mph)	3D arc fitted between hit and bounce using physics and net height	Approximate only	3

Skip for now: forced vs unforced errors (judges disagree on these too), spin, and automatic line calls on balls that touch the line. Show errors as "net" and "out", and flag bounces within ~6 in of a line as "close call" for review.

Target architecture

The event log sits between the pipeline and everything after it. Stats code, and later the app, can be built and tested on hand-labelled logs before the models are finished.

Each video becomes one event log; every stat is computed from it



System architecture · app, event log, CV pipeline

Suggested stack, chosen for a two-person team:

- **CV pipeline:** Python 3.11, PyTorch 2.x, Ultralytics YOLO for people and pose, ByteTrack for tracking, the DinkData TrackNet model for the ball. Ship it as one Docker image.
- **GPU compute:** your old PC as the processing machine for P0–P2, if it has an NVIDIA card with 6 GB+ of memory. Add serverless GPUs (e.g. Modal or RunPod) when wait times grow and for heavy training runs. Skip frames between rallies to cut cost.
- **Web app:** From P3: Next.js, Postgres, S3-compatible storage (e.g. Cloudflare R2), and a job queue. The stats code lives in its own tested package.
- **Labelling:** CVAT, which DinkData already used, for ball points and event tags.

The event log: your contract

Agree on this file format in week 1 and freeze it as v1. The pipeline only has to produce it, and the stats package (later the app) only has to read it. Label the gold set in this same format, so stats code can be tested before the models are finished.

```
{
  "schema": "dinkdata.events/v1",
  "video": {"id": "vid_123", "fps": 30, "width": 1920, "height": 1080},
  "court": {"homography": [[...],[...],[...]], "units": "ft", "source":
"auto|manual"},
  "players": [
    {"id": "p1", "team": "A", "label": "Player 1", "track_ids": [3, 17]}
  ],
  "rallies": [
    {
      "id": "r7", "start_frame": 4210, "end_frame": 4655,
      "server": "p1", "server_number": 2, "score_before": "4-3-2",
      "winner_team": "B", "end_reason":
"net|out|double_bounce|unreturned|fault",
      "events": [
        {"t": 4212, "type": "hit", "player": "p1", "shot": "serve", "volley":
false, "conf": 0.97},
        {"t": 4251, "type": "bounce", "court_xy": [14.2, 37.5], "in": true,
"close_call": false, "conf": 0.93},
        {"t": 4263, "type": "hit", "player": "p3", "shot": "return",
"volley": false, "conf": 0.88}
      ],
      "reviewed": false
    }
  ],
  "tracks": {"ball": "s3://.../ball.parquet", "players":
"s3://.../players.parquet"}
}
```

Rules that keep it useful:

- Every event carries `conf`. The review screen shows low-confidence events first.
- Human edits are saved as a separate patch on top of the model output, never by overwriting it. Each correction becomes new training data.
- Stats are a pure function `events → stats` in one package with unit tests. They are recomputed whenever an event is edited, never stored by hand.

Roadmap

Both of you build the pipeline for the first 14 weeks: ball and court (A) beside players and events (B). The web app starts in P3. Measure each gate on the court you never trained on.

Both partners build the CV pipeline first; the web app waits for P3

Phase	P0 Foundations Weeks 1–3	P1 Rallies Weeks 4–8	P2 Shot stats Weeks 9–14	P3 Web app Week 15 on
Together	Repo, CI, schema v1 10-game gold set Eval harness	Stat definitions Overlay video renderer	Concierge beta: 10 players get stats reports	Launch the app Pricing
Partner A Ball + court	Ball tracker cleanup Ball benchmark Court + kitchen calibration	Bounce detection In/out, close calls Rally segmenter v2	Shot features Drop/drive/lob/dink Net + out errors Auto court lines	Worker + API Approx. ball speed Cloud GPU overflow
Partner B Players + events	Old PC worker setup Player detection + tracking, 4 IDs	Hit detection Who hit it Score tracker Stats package v1	Wrist pose Volley detection Kitchen arrival Retraining loop	Frontend screens Forehand/backhand Speed-ups
◇ **Gate**	Ball recall ⩾ 90% Player IDs ⩾ 95%	Rally F1 ⩾ 95% Hit, bounce F1 ⩾ 90% Winner right ⩾ 90%	Who-hit ⩾ 90% Shot type ⩾ 85% Beta keeps using it	Paying users Review under 10 min

Roadmap · 4 phases, about 14 weeks to beta

The gate numbers are proposed starting targets. Reset them after the first benchmark in P0. Until a stat passes its gate, show it only after human review or behind a "beta" label.

Action plan

Both of you work on the CV pipeline for the first ~14 weeks, split by what you track. Partner A owns the **ball and the court**. Partner B owns the **players and the events** built from them. The web app waits until P3; until then, results go out as overlay videos and stats reports, which is enough for a small beta.

You hand work to each other through four files. Agree on them in week 1 so neither of you waits on the other:

File	Made by	Used by	What it holds
`court.json`	A	B	Homography and court keypoints, including both kitchen lines and the net, per camera
`ball.parquet`	A	B	Per frame: ball x, y, visible, confidence. Per bounce: court x, y, in or out, close call
`players.parquet`	B	A	Per frame: player id, team, box, foot position on court, wrist points
`events.json` (event log v1)	B, with A's bounces and shot types	Stats package, later the app	Rallies, hits, bounces, shot types, winners, score

Together

- [ ] **P0** One repo with `pipeline/court`, `pipeline/ball`, `pipeline/players`, `pipeline/events` and `stats/`, plus tests and CI.
- [ ] **P0** Freeze the four file formats above and event schema v1.
- [ ] **P0** Collect 20+ full PickleCam games from at least 3 courts. Keep 1 court entirely for testing.
- [ ] **P0** Label the 10-game gold set (rally start/end, hit + player, bounce + in/out, point winner). About 1–2 hours per game; split the games.
- [ ] **P0** Eval harness: one command scores every stage against the gold set on the held-out court. Run it before every merge.
- [ ] **P1** One-page definition for every stat, e.g. exactly when a third-shot drop counts as a success.
- [ ] **P1** Overlay video renderer: court lines, ball, player IDs and events drawn on the video. It is your debugging tool and your demo until the app exists.
- [ ] **P2** Concierge beta: process games for 10 players and send each a stats report. Ask which stats they actually use.
- [ ] **P2** Run the same games through PB Vision to compare.

Partner A: ball and court

- [ ] **P0** Recover checkpoints and labels. Repackage tracking as `pipeline/ball` (pathlib, torch 2.x, no import-time file loads). Done when `dink-cv ball clip.mp4` writes a ball track on a Mac and on the old PC.
- [ ] **P0** Benchmark ball tracking on the held-out court: precision, recall, pixel error.
- [ ] **P0** Court calibration: 4 corner clicks fit the full template, so kitchen lines and the net come for free. Cache it per PickleCam camera. Done when projected lines sit on the painted lines within a few pixels.
- [ ] **P1** Bounce detection from the ball track: direction flips first, then a small model trained on the gold set. Done when bounce F1 ≥ 90%.
- [ ] **P1** In/out calls from each bounce's court position; flag bounces within ~6 in of a line as close calls.
- [ ] **P1** Rally segmenter v2 in court units, with the net taken from the homography instead of a pixel band.
- [ ] **P2** Shot features (speed, arc, depth, kitchen landing) and a classifier for drop, drive, lob and dink. Net and out error types.
- [ ] **P2** Automatic court-line detection, so calibration needs no clicks.
- [ ] **P3** Processing worker and API for the app; approximate ball speed from 3D arc fitting; cloud GPU overflow.

Partner B: players and events

- [ ] **P0** Set up the old PC as the processing machine (Linux or Docker, CUDA, PyTorch). Done when it runs the ball tracker on a full game.
- [ ] **P0** Player detection and tracking (YOLO + ByteTrack). Keep the 4 people whose feet are inside the court, using A's `court.json`. Done when player IDs are right in ≥ 95% of frames on the held-out court.
- [ ] **P1** Hit detection and attribution: a ball direction change near a player, assigned to the nearest player. Done when hit F1 ≥ 90%.
- [ ] **P1** Score tracker: a side-out scoring state machine that checks rally winners and server order.
- [ ] **P1** Stats package v1: pure `events → stats` functions with unit tests from the stat definitions. Assemble `events.json`.
- [ ] **P2** Wrist pose to improve who-hit-it; volley vs groundstroke.
- [ ] **P2** Kitchen arrival and court positioning, from player feet against A's kitchen lines.

- [ ] **P2** Retraining loop: turn corrected events into new training data every 2 weeks.
- [ ] **P3** Web app frontend: upload, setup, review screen, stats dashboard. Forehand vs backhand and speed-ups.

Open questions

- [ ] Does anyone still have the TrackNet checkpoints, the CVAT labels and the source videos? If not, P0 starts with relabelling ball positions.
- [ ] What does PickleCam footage look like: resolution, frame rate, mount height and position? I could not find public specs. The plan assumes 1080p, 30 fps, behind one baseline. At 60 fps, hit and bounce detection get much easier.
- [ ] Is the camera the same for every court at a venue? If yes, calibrate once per camera and reuse it.
- [ ] Doubles only, or singles too? Singles is simpler (2 players) and a good first target.
- [ ] Can you get PickleCam's permission or an export API to pull videos directly, instead of users uploading files?
- [ ] Who are the first 10 users (club, coach, league)? Their top 3 stats should decide the Phase 2 order.

Sources

- roberto-cantero/DinkData at commit `e808fd6` (Jul 19, 2025), read in full
- qaz812345/TrackNetV3, the upstream model
- PB Vision app listing, competitor feature list (marketing copy, not verified)
- SportAI video requirements, camera guidance (1080p, 30 fps constant, mounted high on the short side)